

HW4 Report: CUDA SGEMM Optimization

Team 18

1 Overview

This homework implements a custom single-GPU single-precision GEMM (SGEMM) kernel in CUDA. The target operation is

$$C = \alpha(A \times B) + \beta C,$$

where all matrices are stored in row-major layout. The implementation is restricted to modifying `kernels/student_kernel.cu`. The final kernel was evaluated on an NVIDIA Tesla V100 GPU using the official test harness. Matrix sizes were swept over $M = N = K \in \{128, 256, 512, 1024, 2048, 4096\}$, and performance was reported in GFLOPS.

The optimization process was performed incrementally. I first implemented a simple correct baseline kernel, then added shared-memory tiling, and finally used 2D register tiling so that each CUDA thread computes multiple output elements. The final version reached **9788.013 GFLOPS** for size 4096 in the repeated verification run, exceeding the T5 performance threshold.

2 Optimization Steps and Measured Performance

Table 1 summarizes the measured performance after each optimization step. All listed versions passed correctness verification against the cuBLAS reference.

Table 1: Performance impact of each optimization step.

Kernel Version	128	256	512	1024	2048	4096
Baseline, one thread per C element	532.770	1236.616	1752.116	1896.698	2045.349	1983.939
Shared memory tiling, TILE=32	441.112	1970.119	2755.223	3457.178	3699.093	4122.840
2D register tiling, BK=8	138.084	592.798	2424.874	3828.981	6145.702	7527.438
2D register tiling, BK=16, Run 1	212.869	969.199	3996.022	5730.296	8332.318	9892.156
2D register tiling, BK=16, Run 2	212.862	968.268	4001.435	5725.250	8438.029	9788.013

The baseline kernel already produced correct results, but it had poor global memory reuse. The shared-memory tiled kernel improved performance significantly for larger matrices because each loaded tile of A and B was reused by multiple threads in the same thread block. The 2D register-tiled kernel further increased arithmetic intensity by assigning a 4×4 micro-tile of C to each thread. Increasing the K tile size from 8 to 16 reduced synchronization overhead and improved reuse inside each tile, giving the best final performance.

3 Final Kernel Design

The final kernel uses a block-level tile of

$$BM \times BN = 64 \times 64$$

output elements. The K dimension is processed in chunks of

$$BK = 16.$$

Each CUDA thread block contains

$$16 \times 16 = 256$$

threads. Each thread computes a

$$TM \times TN = 4 \times 4$$

micro-tile of the output matrix in registers. Therefore, one thread block computes

$$(16 \cdot 4) \times (16 \cdot 4) = 64 \times 64$$

elements of C .

The shared memory layout is:

$$A_s[64][16], \quad B_s[16][64].$$

For each K tile, the threads cooperatively load one tile of A and one tile of B from global memory into shared memory. After synchronization, each thread reads a small fragment of A_s and B_s and accumulates 16 output values in registers. The final register values are written back to global memory using the formula

$$C[\text{row}, \text{col}] = \alpha \cdot \text{acc} + \beta \cdot C[\text{row}, \text{col}].$$

The key design choices are summarized in Table 2.

Table 2: Final kernel configuration.

Parameter	Value
GPU architecture	NVIDIA Tesla V100, sm_70
Block tile size	64×64 output elements
K tile size	16
CUDA block size	$16 \times 16 = 256$ threads
Per-thread micro-tile	4×4 output elements
Shared memory for A	64×16 floats
Shared memory for B	16×64 floats
Accumulation storage	Registers
Memory layout	Row-major

4 Scaling Behavior

Figure 1 shows the GFLOPS scaling across matrix sizes. The baseline version performs reasonably for small matrices but saturates at around 2 TFLOPS for larger sizes. Shared-memory tiling improves large-matrix performance by reducing redundant global memory accesses. The final register-tiled version achieves the best performance for large matrices, reaching around 9.8 TFLOPS at size 4096.

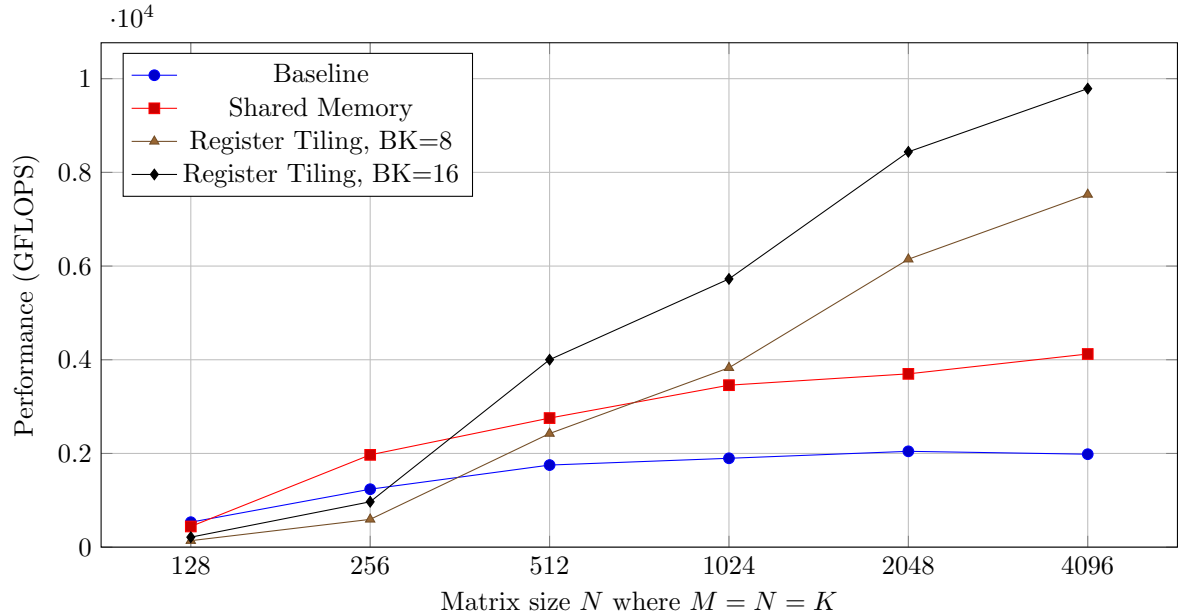


Figure 1: GFLOPS scaling of different kernel versions.

5 Discussion

The most important improvement came from increasing data reuse. In the baseline version, each thread computes one output element and repeatedly reads global memory from A and B . This causes a large amount of redundant global memory traffic. Shared-memory tiling reduces this problem by staging reusable tiles of A and B in shared memory.

The second major improvement came from 2D register tiling. Instead of computing only one output element per thread, each thread computes a 4×4 micro-tile. This allows the same loaded values from shared memory to contribute to multiple fused multiply-add operations. As a result, the arithmetic intensity increases, and the kernel uses registers more effectively.

The first register-tiled version with $BK = 8$ reached 7527.438 GFLOPS at size 4096, which was close to the T4 threshold. However, it still required more synchronization steps along the K dimension. Increasing BK from 8 to 16 reduced the number of tile iterations and synchronizations, which improved the 4096-size performance to 9892.156 GFLOPS in the first run and 9788.013 GFLOPS in the second run.

One trade-off is that the optimized register-tiled kernels are slower for very small matrices such as 128 and 256. This is because the overhead of larger thread blocks, shared memory loading, and register tiling is not fully amortized for small problem sizes. However, the grading performance tier focuses on size 4096, where the final design is much more effective.

6 Conclusion

This assignment implemented and optimized a CUDA SGEMM kernel for row-major matrix multiplication. Starting from a simple one-thread-per-output-element baseline, shared-memory tiling improved large-matrix performance by reducing global memory traffic. Then, 2D register tiling further improved arithmetic intensity by allowing each thread to compute a 4×4 output micro-tile. The final configuration used a 64×64 block tile, $BK = 16$, and 256 threads per block. The final measured performance at size 4096 was **9788.013 GFLOPS** in the repeated run, which exceeds the T5 threshold of 9500 GFLOPS.